

**DISCOVERY PHASE – COMPLETE** (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

**Core functionality required:**

- Assess whether turning one of your rigs into a **Droid Computer** (persistent machine for remotely orchestrating Droids, as announced in the X post) provides meaningful gains for the current Sovereign Machina tri/quad-node stack.
- Focus on standalone personal intelligence use (voice-first assistant, knowledge OS, decision support, life logging, self-improving agentic workflows).
- Maintain: Node A = Mooncake KV buffer, Node B = Director, Node C = Oracle / Voice Core, optional Hermes Pod. No changes to Node A or Node B core logic. Zero-trust, physical sovereignty, local-first design.

All requirements are now exhaustive and production-locked.

**RESEARCH PHASE – RIGOROUS CROSS-CHECK (X post + Droid ecosystem + Sovereign Machina alignment)**

The X post from @FactoryAI announces **Droid Computers**: persistent, stateful dev environments for their Droid agents.

Key features from the post and thread:

- Each Droid Computer has its own filesystem, credentials, and configurations.
- Two modes: Factory-managed cloud instances or **Bring Your Own Machine (BYOM)** — register any laptop, Mac Mini, or VM as a Droid Computer.
- Enables remote orchestration from phone, overnight task queuing, always-on agents that triage alerts and run deep research.
- Sessions build on each other, so agents “get smarter the more they work.”
- Designed for complex, stateful software engineering workflows beyond simple one-off tasks.

**DROID COMPUTER IMPLICATIONS FOR YOUR SOVEREIGN MACHINA STACK**

**Potential Gains**

## 1. Persistent, Stateful Agent Runtime

- Your current Hermes + GBrain plans provide strong metacognitive orchestration, but a Droid Computer adds a ready-made, persistent filesystem + credential store for long-running agents.
- Example: A Droid agent could run overnight background research, triage voice notes from Omi, or monitor RKG changes — all with persistent state that survives reboots.
- This complements (does not replace) Hermes: Droid handles execution state, Hermes handles enrichment, auditing, and self-healing.

## 2. Remote Orchestration & Mobility

- BYOM mode lets you register one rig (e.g., the Hermes Pod or a spare machine) as a Droid Computer. You could then control it from your phone via the Factory app or CLI without needing a full VPN tunnel for every interaction.
- Useful for “queue tasks while at work” — speak a voice note in Machina Terminal, it syncs home, and the Droid Computer executes it in the background.

## 3. Always-On Background Agents

- Droid Computers are explicitly built for “always-on agents that triage alerts and run deep research.” This could offload some background tasks from your Hermes Pod, freeing Node C for real-time voice and Node B for reasoning.

## Honest Trade-offs & Risks

- **Sovereignty:** BYOM mode keeps the machine local, but the orchestration layer (Factory’s control plane) may involve some cloud coordination. This is a partial sovereignty compromise compared to your fully local MCP/VSB + Hermes setup.
- **Overlap:** Much of what Droid Computers provide (persistent state, credential management, background execution) is already being built into your Hermes Pod + GBrain rewrite. Adding Droid would create redundancy unless you treat it as a specialized execution runtime.

- **Maintenance & Complexity:** You would now have two agent frameworks (your native Hermes/GBrain + Factory's Droid) to manage. This increases cognitive load.
- **Performance:** Droid Computers are optimized for software engineering workflows. Your stack is optimized for personal intelligence (voice, RKG, self-healing). The two may not mesh perfectly without custom bridging.

## Verdict

You stand to gain **modest convenience** in remote orchestration and persistent background task execution, especially if you use BYOM on a spare rig. However, the gains are **not transformative** because your existing Hermes + GBrain plans already target similar capabilities in a fully sovereign way. The main benefit is faster “always-on agent” experimentation via Factory's polished runtime, at the cost of some sovereignty and added complexity.

## Recommended Approach

- **Short-term:** Do **not** convert a core rig yet. Use a spare machine or VM as a test Droid Computer via BYOM to evaluate the workflow.
- **Long-term:** If the convenience proves valuable, integrate Droid as a specialized execution runtime on the Hermes Pod (via MCP bridge), keeping your native Hermes/GBrain as the primary metacognitive layer.
- **Sovereignty-First Path:** Continue building the native Rust/Go GBrain rewrite first — it will likely deliver 80–90% of the Droid Computer benefits without external dependencies.

## ARCHITECT SIGN-OFF

Turning one rig into a Droid Computer provides modest gains in persistent state management and remote orchestration convenience, but it is not a game-changer for your stack. Your existing Hermes + planned GBrain rewrite already targets similar capabilities in a fully sovereign, local-first manner. The main value of Droid Computers is polished background agent execution and BYOM simplicity. Use it as an experimental sidecar on a spare machine if you want to test the workflow, but prioritize the native rewrite for long-term sovereignty. This keeps your tri/quad-node architecture clean and under your complete control.

Your main sovereign machine remains completely untouched and rock-solid.

**Clarifying questions (mandatory before next phase):**

1. Do you want a concrete MCP bridge design to treat a Droid Computer as an execution runtime for Hermes/GBrain?
2. Shall I outline a test setup for registering a spare rig as a BYOM Droid Computer?

Reply with your priority and I will deliver the concrete design. The Droid Computer evaluation is complete.